



Intoduction To Programming Using C++ Level 2 Recursion

The background of the slide is a photograph of a modern building's interior. It features a large, curved glass ceiling that allows natural light to filter through, creating a pattern of light and shadow on the floor. In the lower part of the image, the silhouettes of several people are visible, some standing and some sitting, suggesting a collaborative workspace or a public area. The overall color palette is muted, with greys, blues, and the warm tones of the light coming through the glass.

Introduction To Programming Using C++

Level 2

Recursion

Definition

- ◆ Recursion is a technique where a Function calls itself from within its own body.
- ◆ It is used to repeat an operation in a structured way without necessarily using traditional loops such as `for` or `while`.
- ◆ A Base Case is essential because it defines when the Recursive Calls must stop.

Explanation

- A Recursive Function usually consists of two important parts:
 - ▨ Base Case → The condition that stops the Recursion.
 - ▨ Recursive Case → The part where the Function calls itself with a new value.
- Each Recursive Call creates a new Stack Frame in the Call Stack.
- The current Function Call waits for the next Recursive Call to finish before it can continue.
- Because the Call Stack follows Last In, First Out (LIFO), the last Recursive Call is the first one to complete when the Functions start returning.

Procedures

1. Define the Function.
2. Add a Base Case to determine when the Recursion stops.
3. Add the Recursive Call.
4. Make sure each Recursive Call moves closer to the Base Case.
5. When the Base Case is reached, the Recursive Calls stop.

💡 Example

```
void PrintNumbersFromNtoM(int N, int M)
{
    if (N <= M)
    {
        cout << N << endl;
        PrintNumbersFromNtoM(N + 1, M);
    }
}

int main()
{
    PrintNumbersFromNtoM(1, 10);

    return 0;
}
```

➡ Result

1

2

3

.

.

10

Note

- ◆ The Base Case in this example is the condition:
if ($N \leq M$)
- ◆ More precisely, the Recursion stops when ' $N > M$ '.
- ◆ Each Recursive Call must move toward the stopping condition. Here, ' N ' increases by ' 1 ' each time.

⊘ Warning

- ◆ If a Recursive Function never reaches its Base Case, it can continue creating Stack Frames until a Stack Overflow occurs.
- ◆ Recursion can also use significant memory when the number of Recursive Calls becomes very large.
- ◆ For problems where the number of Calls can become very large, an iterative solution using a `for` or `while` loop may be more appropriate.



Comparison

▨ Recursion

- ✓ A Function calls itself.
- ✓ Uses the Call Stack to keep track of Recursive Calls.
- ✓ Requires a Base Case to stop.
- ✓ Useful for problems that naturally have recursive structures, such as tree traversal and certain search algorithms.

▤ Iteration

- ✓ Uses loops such as `for` and `while`.
- ✓ Does not create a new Function Call for every iteration.
- ✓ Can be more memory-efficient for simple repetitive operations.
- ✓ Useful when the number of repetitions can be handled naturally with a loop.

✦✦ Summary

- ✓ Recursion means a Function calls itself.
- ✓ A Base Case is necessary to stop the Recursion.
- ✓ Each Recursive Call creates a new Stack Frame.
- ✓ Recursive Calls follow the Call Stack's LIFO behavior.
- ✓ Each Recursive Call should move closer to the Base Case.
- ✓ Too many Recursive Calls can cause Stack Overflow.

Thank you 😊